

Detalle de lo que hay que lograr en cada sección

Explicación funcional de las 39 tareas del plan de MVP, ordenadas por relevancia: primero las correcciones que pide el tribunal de tesis, y luego el resto de lo que más valor aporta al proyecto a lo que es mejora de detalle o pulido final — sin código, para entender el objetivo antes de programar.

Fuentes: GEF_Secure_Unificado.docx (Plan Unificado de Mejoras) + verificación directa sobre el código fuente del backend y frontend

Generado: 2026-08-02

Orden: por relevancia para el proyecto — no por área técnica (backend/frontend)

Cómo leer cada sección: primero la situación actual (qué pasa hoy en el sistema), después qué se busca lograr, qué implica en términos generales (sin entrar en código), y por qué importa. Donde aplica, se indica también si hay un requisito previo de otra sección — el número de esa sección referenciada ya está actualizado al nuevo orden.

Correcciones que pide el tribunal de tesis

Van antes que cualquier otra tarea: si no están resueltas, cualquier dato o métrica que se genere después queda contaminado.

SECCIÓN 1

BLOQUEANTE DE TESIS

MTTR real de punta a punta

SITUACIÓN ACTUAL

El Dashboard hoy no calcula ni muestra el MTTR (Mean Time To Remediate) en ningún lugar del sistema, a pesar de que es la métrica central que la tesis presenta como resultado clave del proyecto. Además, el dictamen del tribunal señaló que el sistema usa el término "MTTR" para referirse en realidad al tiempo entre detección y notificación, que no es lo mismo que el tiempo hasta la remediación real que exige la literatura citada.

QUÉ SE BUSCA

Calcular el tiempo real que transcurre entre que una vulnerabilidad se detecta y el momento en que efectivamente se marca como resuelta, promediado sobre las vulnerabilidades ya resueltas, y mostrar ese número en el Dashboard como una tarjeta más, con el mismo estilo que las demás métricas existentes.

QUÉ IMPLICA

Un cálculo nuevo del lado del servidor que recorra las vulnerabilidades resueltas y promedie la diferencia entre su fecha de detección y su fecha de resolución. Ese valor tiene que llegar hasta la pantalla del Dashboard y mostrarse como una tarjeta visual más.

POR QUÉ IMPORTA

Es el KPI que la tesis usa para demostrar que el producto ayuda a gestionar el tiempo de respuesta del equipo de seguridad — sin este número, la propuesta de valor central del proyecto queda sin evidencia concreta. Resolverlo correctamente además evita tener que explicarle al tribunal por qué se usó mal el término.

Requisito previo: para que el promedio tenga sentido tiene que haber al menos alguna vulnerabilidad marcada como resuelta, y para que esa marca sea confiable primero tiene que estar funcionando la validación de transiciones de estado (Sección 2).

Ciclo de vida de vulnerabilidades con validación real

SITUACIÓN ACTUAL

Hoy el sistema acepta cualquier cambio de estado sobre una vulnerabilidad sin validar nada — se puede mover una tarjeta directamente de "Detectada" a "Resuelta" en el Kanban sin pasar por "En análisis", sin que el sistema lo impida ni avise.

QUÉ SE BUSCA

Definir cuáles son los cambios de estado permitidos dentro del ciclo de vida (Detectada → En análisis → Resuelta, con la posibilidad de volver de En análisis a Detectada, o de Resuelta nuevamente a En análisis si hace falta reabrir el caso) y rechazar cualquier otro movimiento que no respete ese camino.

QUÉ IMPLICA

Que el sistema identifique un intento de cambio de estado inválido y lo rechace con un aviso claro, en lugar de guardarlo silenciosamente. Del lado del Kanban, si alguien intenta arrastrar una tarjeta a una columna a la que todavía no puede pasar, la tarjeta tiene que volver a su lugar y mostrar un mensaje de error, en vez de quedar "movida" como si nada hubiera pasado.

POR QUÉ IMPORTA

Cualquier número que se calcule después sobre el ciclo de vida de las vulnerabilidades —empezando por el MTTR de la Sección 1— depende de que los estados por los que pasa una vulnerabilidad sean reales y confiables. Si se puede saltar pasos, cualquier métrica queda contaminada. El propio dictamen del tribunal marca esto como la base a corregir antes de confiar en cualquier medición posterior, por eso conviene resolverla primero de toda la lista.

El resto del plan, ordenado por importancia para el proyecto

A partir de acá ninguna tarea es bloqueante para la tesis. Están ordenadas de la que más valor aporta al proyecto a la que es mejora de detalle o pulido final — no por área técnica.

SECCIÓN 3

BACK + FRONT

Escaneo confiable y no bloqueante

SITUACIÓN ACTUAL

Se revisó el código real y se confirmaron tres problemas concretos en la función que dispara un escaneo desde el botón del Inventario: si el llamado al motor de automatización falla por cualquier motivo, el sistema lo registra en un log pero nunca informa el error hacia arriba — el usuario siempre ve un mensaje de éxito, haya funcionado o no. Además, la misma operación consulta el mismo activo en la base de datos dos veces sin necesidad. Y el llamado al motor de automatización es una espera bloqueante: si tarda unos segundos en responder, la pantalla del usuario queda esperando ese tiempo sin poder hacer nada más.

QUÉ SE BUSCA

Que un error real en el disparo del escaneo efectivamente llegue como error hasta el usuario (en vez de mostrarle éxito), eliminar la consulta duplicada, y que el llamado al motor de automatización se dispare en segundo plano para que el botón responda de inmediato en vez de dejar la pantalla esperando.

QUÉ IMPLICA

Corregir el manejo de errores del lado del servidor para que se propaguen en vez de ocultarse, simplificar la operación para que no repita la misma consulta, y hacer que el llamado no bloquee la respuesta. Del lado de la pantalla, el manejo del error para mostrar el mensaje correcto ya está implementado — lo que falta es que, una vez corregido el servidor, ese mensaje refleje lo que realmente pasó.

POR QUÉ IMPORTA

Es el feature más visible del producto — el botón de escanear— y hoy puede fallar en silencio delante de cualquiera que lo esté probando, sin que nadie se entere. Es un problema de confianza más que de funcionalidad: el sistema puede estar mintiendo sobre su propio estado.

Corrección del filtro de fecha por defecto en el Kanban

SITUACIÓN ACTUAL

El filtro de fecha del Kanban arranca con el valor de hoy en vez de vacío. El propio código ya contempla que un filtro vacío significa "mostrar todas las vulnerabilidades sin filtrar por fecha", pero el valor inicial nunca llega a estar vacío — siempre arranca en la fecha actual.

QUÉ SE BUSCA

Que el Kanban arranque sin ningún filtro de fecha aplicado, mostrando todas las vulnerabilidades activas desde el primer momento en que se abre la página.

QUÉ IMPLICA

Es un cambio mínimo del lado de la pantalla — cambiar el valor inicial de ese filtro de la fecha de hoy a vacío.

POR QUÉ IMPORTA

Es el tipo de bug que se nota inmediatamente en cualquier demostración: cualquiera que abra el Kanban por primera vez ve un tablero completamente vacío, aunque el sistema tenga vulnerabilidades activas cargadas, hasta que se le ocurra tocar el filtro de fecha. Es una corrección de esfuerzo mínimo con un impacto muy alto en la primera impresión del producto.

Seguridad con Spring Security

SITUACIÓN ACTUAL

El sistema no tiene ningún tipo de autenticación. Cualquiera que llegue a la dirección del sistema puede ver y modificar todo sin identificarse. La tabla de usuarios existe, pero solo se usa para asignar vulnerabilidades en el Kanban — no tiene contraseña ni ningún mecanismo de login.

QUÉ SE BUSCA

Que el sistema pida usuario y contraseña para poder entrar, y que existan dos tipos de usuario con permisos distintos: uno que puede hacer cualquier operación (Administrador) y otro que solo puede consultar información sin poder modificar nada (Auditor).

QUÉ IMPLICA

Del lado del servidor, un mecanismo de login que valide credenciales e identifique en cada operación quién la está pidiendo y qué le está permitido hacer según su rol, bloqueando las acciones que no correspondan. Del lado de la pantalla, una pantalla de inicio de sesión, y ocultar o deshabilitar las acciones que el usuario actual no tiene permitido ejecutar según su rol.

POR QUÉ IMPORTA

Es la funcionalidad que separa a un prototipo académico de un sistema utilizable por una organización real con más de una persona operándolo — sin esto no hay forma de saber quién hizo qué, ni de limitar lo que cada persona puede tocar.

Gestión de entornos (CRUD real)

SITUACIÓN ACTUAL

El backend ya tiene un controlador completo para entornos de negocio, con creación, edición y borrado — pero se confirmó que el frontend nunca lo usa para eso: solo trae la lista de entornos existentes para mostrarla como opción dentro del formulario de un activo. No hay ninguna pantalla donde se pueda dar de alta un entorno nuevo, editarlo o borrarlo. Además, el campo de criticidad de cada entorno se carga como texto libre, lo que ya generó inconsistencias reales — el código del frontend tiene que contemplar a mano tanto la versión en español como en inglés de la misma criticidad porque los datos cargados no son uniformes.

QUÉ SE BUSCA

Una pantalla propia de gestión de entornos donde se puedan crear, editar y borrar entornos de negocio, y donde la criticidad se elija de una lista fija de opciones en vez de escribirse a mano.

QUÉ IMPLICA

Del lado del servidor, no hace falta nada nuevo — el punto de acceso para crear, editar y borrar entornos ya existe y funciona. Del lado de la pantalla, construir una vista nueva (podría vivir dentro de la sección de Configuración, igual que la gestión de usuarios) con el listado de entornos y los formularios correspondientes, reemplazando el campo de texto libre de criticidad por un desplegable con las opciones válidas.

POR QUÉ IMPORTA

Es un caso particular donde el trabajo de backend ya está hecho y nunca se conectó — completar esta sección es, en gran parte, cablear una pantalla nueva sobre algo que ya funciona. Además, resuelve de raíz un problema de datos que ya se nota en el código: si la criticidad se elige de una lista fija en vez de escribirse libre, cualquier funcionalidad que dependa de ese valor (el semáforo del inventario, la vista de riesgo por entorno) deja de depender de que alguien haya escrito el mismo texto de la misma forma cada vez.

Vista de riesgo por entorno

SITUACIÓN ACTUAL

Los entornos de negocio (Producción, Testing, Desarrollo, etc.) existen en el sistema y se usan para asignarle criticidad a cada activo, pero no tienen ninguna pantalla propia — solo aparecen como una opción dentro del formulario para crear o editar un activo. No hay forma de ver, agrupado por entorno, cuántos activos y cuántas vulnerabilidades tiene cada uno.

QUÉ SE BUSCA

Una vista dedicada a los entornos donde se pueda ver, para cada uno, cuántos activos contiene, cuántas vulnerabilidades abiertas tiene en total y cómo se distribuyen por prioridad — permitiendo comparar de un vistazo qué tan expuesto está Producción respecto a Testing o Desarrollo.

QUÉ IMPLICA

Del lado del servidor, agregar consultas que agrupen las vulnerabilidades y los activos por entorno. Del lado de la pantalla, una sección o página nueva que muestre esa información, idealmente con algún gráfico comparativo entre entornos.

POR QUÉ IMPORTA

La idea central del producto es que la misma vulnerabilidad técnica pese distinto según la criticidad del entorno donde vive — hoy esa idea está resuelta en el cálculo automático de prioridad, pero no hay ninguna pantalla que la haga visible de forma comparativa. Esta vista convertiría el diferenciador principal del proyecto en algo que se puede mostrar y no solo explicar.

Corrección de tipos de prioridad en el frontend

SITUACIÓN ACTUAL

El tipo que declara los valores posibles de prioridad de una vulnerabilidad está escrito en español ('CRITICA', 'ALTA', 'MEDIA', 'BAJA'), pero los datos reales que devuelve el sistema y que ya usa el propio Dashboard para pintar los colores de severidad están en inglés ('CRITICAL', 'HIGH', 'MEDIUM', 'LOW'). El código funciona hoy porque en varios lugares se contempla el valor real como texto libre, pero la definición de tipo no refleja la realidad.

QUÉ SE BUSCA

Que el tipo de prioridad declarado en el código coincida con los valores que el sistema realmente usa.

QUÉ IMPLICA

Es un cambio acotado del lado de la pantalla — corregir la definición del tipo, y revisar los lugares que lo usan para confirmar que sigan funcionando con los valores reales.

POR QUÉ IMPORTA

Varias de las funcionalidades ya planeadas dependen de que el valor de prioridad sea siempre consistente — el semáforo del inventario, los filtros avanzados por prioridad, el ranking de activos más riesgosos. Si la definición de tipo no refleja la realidad, cualquiera de esas funcionalidades corre el riesgo de compararse contra el valor equivocado sin que nadie lo note hasta que ya esté en producción.

Inventario inteligente

SITUACIÓN ACTUAL

La página de Inventario muestra la lista de activos (nombre, software, versión, ecosistema, entorno) pero no da ninguna pista visual de qué tan riesgoso es cada uno — para saber si un activo tiene vulnerabilidades abiertas hay que buscarlas por otro lado. Tampoco se puede filtrar la lista más que por texto libre.

QUÉ SE BUSCA

Que cada fila del inventario muestre cuántas vulnerabilidades abiertas tiene ese activo y cuál es la más grave entre ellas, representado con un semáforo de color (rojo si tiene algo crítico, amarillo si lo más grave es alto, verde si está limpio). Además, poder filtrar la lista completa por ecosistema (npm, pip, maven, etc.) y por entorno de negocio (Producción, Testing, Desarrollo), con botones clicables arriba de la tabla, sin escribir nada.

QUÉ IMPLICA

Del lado del servidor, calcular para cada activo cuántas vulnerabilidades tiene actualmente abiertas y cuál es la de mayor prioridad, y permitir que la búsqueda de activos acepte esos dos filtros nuevos además del texto libre existente. Del lado de la pantalla, agregar la columna con el conteo y el semáforo, y los botones de filtro rápido.

POR QUÉ IMPORTA

Hoy un analista tiene que entrar activo por activo para saber si algo necesita atención. Con esta mejora, el estado de riesgo de todo el inventario se lee de un vistazo — exactamente el tipo de visibilidad centralizada que el proyecto promete resolver.

Dashboard: nuevos indicadores y alertas (KEV, variación semanal, banner, progreso)

SITUACIÓN ACTUAL

El Dashboard muestra números totales (activos, vulnerabilidades, críticas, resueltas este mes) y algunos gráficos, pero todos son valores estáticos de "foto actual". No dicen si la situación mejoró o empeoró respecto a la semana pasada, no llaman la atención sobre algo urgente sin leer los gráficos con cuidado, y tampoco muestran cuántas vulnerabilidades están confirmadas como explotadas activamente — ese dato existe en cada vulnerabilidad pero hoy solo se usa mezclado dentro del cálculo de prioridad.

QUÉ SE BUSCA

Sumar varias mejoras sobre la misma página del Dashboard: una tarjeta dedicada que muestre cuántas vulnerabilidades abiertas están confirmadas como explotadas activamente (KEV); mostrar la variación de cada métrica respecto a la semana anterior (por ejemplo "+12 esta semana" o "-10 desde la semana pasada"); un banner de alerta en la parte superior de la página cuando existan vulnerabilidades críticas todavía abiertas, con acceso directo al Kanban; y una barra de progreso que muestre qué porcentaje del total de vulnerabilidades ya fue resuelto.

QUÉ IMPLICA

Del lado del servidor, contar las vulnerabilidades abiertas confirmadas como explotadas, comparar la cantidad de vulnerabilidades detectadas y resueltas en los últimos 7 días contra los 7 días anteriores, y sumar todo esto a las estadísticas que ya devuelve el Dashboard. Del lado de la pantalla, una tarjeta nueva para la explotación activa, la variación junto a cada tarjeta de métrica existente, el banner de alerta cuando corresponda, y la barra de progreso en un lugar visible.

POR QUÉ IMPORTA

Convierte al Dashboard de una foto fija a algo que comunica tendencia y urgencia sin que el usuario tenga que interpretar nada. Además, la explotación activa confirmada es uno de los tres factores centrales del motor de riesgo del producto, junto con el CVSS y la criticidad del entorno — hoy es el único de los tres que no tiene ningún número propio visible en el Dashboard. Estas mejoras van juntas en una sola sección porque tocan exactamente los mismos archivos del lado del servidor y de la pantalla.

Panel de Configuración con acceso restringido a Administradores

SITUACIÓN ACTUAL

La página de Configuración ya existe y hoy tiene dos pestañas — gestión de usuarios y log de errores del sistema — pero cualquiera que entre a la aplicación puede verla y usarla, porque no hay ningún control de acceso por rol (de hecho, hoy no hay ningún tipo de login). Además, la gestión de entornos (Sección 6) todavía no tiene ningún lugar dentro de la interfaz.

QUÉ SE BUSCA

Reorganizar todas las herramientas de administración del sistema —gestión de usuarios, gestión de entornos y el log de errores— dentro de una única sección de Configuración, y que esa sección completa quede visible y accesible solo para el rol Administrador. Un usuario con rol Auditor no debería ni ver la opción de Configuración en el menú.

QUÉ IMPLICA

Del lado del servidor, además de tener el login y los roles funcionando (Sección 5), hay que asegurarse de que cada punto de acceso relacionado con usuarios, entornos y errores del sistema rechace la operación si quien la pide no tiene rol Administrador — no alcanza con ocultar el botón en la pantalla, el control tiene que existir también del lado del servidor para que no se pueda saltar. Del lado de la pantalla, agrupar dentro de la misma sección de Configuración las pestañas de Usuarios, Entornos y Errores del sistema, y ocultar esa sección completa del menú para cualquiera que no tenga el rol correspondiente.

POR QUÉ IMPORTA

Hoy varias de las funcionalidades más sensibles del sistema —dar de alta usuarios, borrar el log de errores, y en el futuro crear o borrar entornos de negocio— están mezcladas con el resto de la aplicación sin ninguna restricción. Consolidarlas en un solo lugar protegido no es solo prolijidad: es la forma de que la seguridad implementada en la Sección 5 tenga un efecto real dentro del producto, en vez de quedar como un login que no cambia lo que cada persona puede hacer una vez adentro.

Requisito previo: depende directamente de que el login y los roles (Sección 5 — Seguridad con Spring Security) estén implementados, y le da a la gestión de entornos (Sección 6) un lugar formal dentro de la interfaz.

Gestión de falsos positivos (whitelist)

SITUACIÓN ACTUAL

Existe una tabla en la base de datos que se llena automáticamente con vulnerabilidades que el sistema decide ignorar porque no corresponden a ningún activo interno registrado — es una lista de "ruido" filtrado. Hoy nadie puede verla ni gestionarla desde la aplicación; solo se puede consultar entrando directamente a la base de datos.

QUÉ SE BUSCA

Poder ver esa lista de vulnerabilidades ignoradas desde la aplicación, y poder eliminar una entrada puntual si se decide que en realidad no debería haber sido ignorada (para que vuelva a considerarse en el próximo escaneo).

QUÉ IMPLICA

Del lado del servidor, exponer esa tabla para poder leerla y borrar una entrada específica. Del lado de la pantalla, una vista donde se liste ese contenido y se pueda accionar sobre cada fila.

POR QUÉ IMPORTA

La tesis menciona explícitamente la gestión de falsos positivos desde la interfaz como un diferenciador frente a herramientas como Nessus — hoy esa promesa no está cumplida porque la lista existe pero es invisible para el usuario.

Tendencia histórica de MTTR

SITUACIÓN ACTUAL

Con el MTTR real ya contemplado (Sección 1), el Dashboard mostraría un único valor: el promedio actual. No habría forma de ver si ese número viene mejorando o empeorando con el tiempo.

QUÉ SE BUSCA

Un gráfico que muestre cómo evolucionó el MTTR semana a semana (o mes a mes), en vez de un único número congelado en el presente.

QUÉ IMPLICA

Del lado del servidor, calcular el MTTR agrupado por período en vez de un solo promedio general. Del lado de la pantalla, un gráfico de tendencia junto a la tarjeta de MTTR ya existente.

POR QUÉ IMPORTA

Un solo número dice cómo está el equipo hoy; una tendencia dice si está mejorando. Para la tesis, es evidencia mucho más fuerte poder mostrar que el tiempo de resolución bajó con el tiempo, no solo cuál es el valor puntual actual.

Historial del caso: notas y auditoría de cambios

SITUACIÓN ACTUAL

Una vulnerabilidad se le puede asignar a una persona, pero no hay ningún lugar donde alguien pueda dejar constancia de por qué tomó una decisión, qué investigó, o qué la llevó a marcarla como resuelta. Tampoco hay ningún registro de quién cambió su estado ni en qué momento — solo se guarda la fecha de la última actualización, sin ningún rastro de quién la hizo.

QUÉ SE BUSCA

Un historial completo dentro del detalle de cada vulnerabilidad que combine dos cosas: comentarios que un analista puede escribir a mano (notas de investigación, justificación de una decisión) y un registro automático e inmutable de cada cambio relevante (quién la asignó, quién cambió su estado, cuándo) — todo junto, como una única línea de tiempo del caso.

QUÉ IMPLICA

Del lado del servidor, dos piezas trabajando en conjunto: un lugar para guardar las notas manuales que alguien decide escribir, y un registro que se genere automáticamente cada vez que se modifica algo relevante de la vulnerabilidad, en vez de sobrescribir directamente el dato anterior. Del lado de la pantalla, un espacio dentro del detalle de la vulnerabilidad (por ejemplo, dentro del modal que ya se abre al hacer clic en una tarjeta) donde se vea esa línea de tiempo combinada: los eventos automáticos intercalados con los comentarios manuales, en orden cronológico.

POR QUÉ IMPORTA

La tesis promete que el sistema gestiona las vulnerabilidades "de forma contextualizada, trazable y auditable" — hoy el sistema registra en qué estado está algo, pero no por qué llegó ahí ni quién hizo qué. Juntar el rastro automático (que no depende de que nadie se acuerde de dejarlo) con las notas manuales (que aportan el contexto humano de la decisión) da una trazabilidad completa en un solo lugar, en vez de dos historiales separados que alguien tendría que cruzar a mano.

Requisito previo: para que tenga sentido registrar "quién" hizo cada cambio, primero tiene que existir un login real (Sección 5 — Seguridad con Spring Security). Hoy nadie se identifica al usar el sistema.

Historial de escaneos (sección nueva)

SITUACIÓN ACTUAL

Cada vez que se ejecuta el pipeline de detección, el resultado de esa ejecución ya queda guardado en la base de datos — pero no existe ninguna forma de verlo desde la aplicación. Hoy esa información solo es accesible entrando directamente a la base de datos.

QUÉ SE BUSCA

Una sección nueva en el sistema donde se pueda ver el historial completo de ejecuciones del pipeline: cuándo corrió cada scan y qué resultado tuvo.

QUÉ IMPLICA

Del lado del servidor, exponer esa información ya existente a través de un punto de acceso que la aplicación pueda consultar, idealmente de a partes para no traer todo el historial de una sola vez. Del lado de la pantalla, una página nueva con una tabla que liste esas ejecuciones de la más reciente a la más vieja.

POR QUÉ IMPORTA

Le da al equipo (y a quien evalúe el proyecto) visibilidad sobre si la automatización funciona de forma consistente en el tiempo, sin depender de mirar logs técnicos. Es, además, una de las mejoras de menor esfuerzo con mayor impacto visual de toda la lista, porque los datos ya existen — solo falta mostrarlos.

Detalle de activo con historial de vulnerabilidades

SITUACIÓN ACTUAL

No existe ninguna vista donde se pueda ver, para un activo puntual, todas las vulnerabilidades que se le detectaron a lo largo del tiempo. Las vulnerabilidades solo se pueden ver todas juntas (listado general) o por su estado actual en el Kanban.

QUÉ SE BUSCA

Que al entrar al detalle de un activo específico en el Inventario, se pueda ver su historial completo de vulnerabilidades detectadas, ordenadas de la más reciente a la más antigua.

QUÉ IMPLICA

Del lado del servidor, un punto de acceso nuevo que, dado un activo, devuelva todas las vulnerabilidades asociadas a él (esa relación ya existe en los datos, solo falta poder consultarla de esa forma). Del lado de la pantalla, una vista expandida o un timeline dentro del Inventario que muestre esa información al hacer clic sobre un activo puntual.

POR QUÉ IMPORTA

Hoy, si un analista quiere entender el historial de riesgo de un activo en particular (por ejemplo, para justificar reemplazarlo), no tiene ninguna vista dedicada para hacerlo — tiene que buscar manualmente entre todas las vulnerabilidades del sistema.

Filtros avanzados en el listado de vulnerabilidades

SITUACIÓN ACTUAL

El listado general de vulnerabilidades solo se puede filtrar por estado (Detectada, En análisis, Resuelta). No se puede filtrar por prioridad, por ecosistema, ni buscar una vulnerabilidad puntual por su identificador CVE.

QUÉ SE BUSCA

Sumar la posibilidad de filtrar ese listado también por prioridad y por ecosistema, y de buscar directamente por el identificador de la vulnerabilidad (CVE o GHSA).

QUÉ IMPLICA

Del lado del servidor, que el punto de acceso que devuelve el listado acepte esos filtros adicionales. Del lado de la pantalla, agregar los controles de filtro correspondientes (desplegables o campo de búsqueda) donde se muestra ese listado.

POR QUÉ IMPORTA

A medida que el volumen de vulnerabilidades detectadas crece, encontrar una en particular o acotar la vista a lo que realmente importa (por ejemplo, "solo las críticas de npm") se vuelve cada vez más necesario. Hoy la única forma de encontrar algo puntual es filtrando por estado y revisando manualmente.

Paginación de listados

SITUACIÓN ACTUAL

El listado general de vulnerabilidades devuelve todos los registros existentes en una sola respuesta, sin límite. Con pocos datos no se nota, pero a medida que el sistema acumula uso real esto empieza a ser un problema de velocidad de carga.

QUÉ SE BUSCA

Que ese listado se entregue de a partes (páginas), en vez de todo junto.

QUÉ IMPLICA

Del lado del servidor, devolver los resultados divididos en páginas de un tamaño razonable, ordenados por fecha de detección. Del lado de la pantalla, agregar los controles para avanzar y retroceder entre esas páginas en cualquier vista que hoy liste vulnerabilidades sin paginar.

POR QUÉ IMPORTA

Es una mejora de rendimiento preventiva — protege al sistema de volverse lento a medida que se acumulan más scans y más vulnerabilidades detectadas con el uso real.

Ranking de activos más riesgosos

SITUACIÓN ACTUAL

El Dashboard muestra totales generales (activos, vulnerabilidades, críticas) pero no hay ninguna vista que diga cuáles activos puntuales concentran más riesgo.

QUÉ SE BUSCA

Una lista con los activos que más vulnerabilidades críticas o de alta prioridad acumulan, ordenados de mayor a menor riesgo.

QUÉ IMPLICA

Del lado del servidor, un cálculo que cuente y ordene los activos según sus vulnerabilidades abiertas más graves. Del lado de la pantalla, una lista o tabla corta (por ejemplo, los primeros 10) en el Dashboard o en una vista propia.

POR QUÉ IMPORTA

Es distinto de la vista por entorno (que agrupa) y del semáforo del Inventario (que muestra el estado de todos los activos por igual) — esto responde directamente a la pregunta "¿en qué activo puntual me tengo que enfocar primero?", que es la que un analista se hace todos los días.

Vencimientos / SLA en vulnerabilidades críticas

SITUACIÓN ACTUAL

Una vulnerabilidad puede quedar "En análisis" indefinidamente sin que el sistema marque ningún límite de tiempo — no hay ningún concepto de plazo o vencimiento asociado a la urgencia de una vulnerabilidad.

QUÉ SE BUSCA

Poder definir una fecha límite de resolución para las vulnerabilidades más críticas, y que el sistema señale claramente cuando ese plazo ya se venció.

QUÉ IMPLICA

Del lado del servidor, guardar esa fecha límite junto con cada vulnerabilidad y calcular si ya está vencida. El cálculo y la carga de esa fecha se resuelven enteramente en el backend de Spring Boot —por ejemplo, calculándola en el momento en que alguien la asigna o la revisa por primera vez—, sin depender de que el workflow de n8n haga nada distinto de lo que ya hace hoy al insertar la vulnerabilidad. Del lado de la pantalla, mostrar ese plazo y resaltar de forma clara las vulnerabilidades que ya lo superaron, tanto en el Kanban como en el listado general.

POR QUÉ IMPORTA

Le da al concepto de "criticidad" una consecuencia concreta en el tiempo — hoy una vulnerabilidad crítica se prioriza visualmente, pero nada le exige al equipo resolverla dentro de un plazo determinado. Es un paso más hacia la idea de "gestión auditable" que plantea la tesis.

Exportación de reportes de vulnerabilidades (CSV y PDF)

SITUACIÓN ACTUAL

No hay ninguna forma de sacar los datos del sistema más que mirándolos en pantalla — no se puede descargar un archivo con la información de las vulnerabilidades para procesarla en otro lado, ni generar un documento prolijo y presentable para compartir con alguien de afuera.

QUÉ SE BUSCA

Dos formas de exportar el mismo tipo de información, cada una pensada para un uso distinto: un archivo CSV con el listado completo de vulnerabilidades (identificador, CVE, activo afectado, prioridad, estado, puntaje CVSS, si está siendo explotada activamente, fecha de detección y de resolución) pensado para abrir en una planilla y procesar los datos; y un reporte en PDF con el mismo tipo de resumen pero en un formato listo para compartir directamente con alguien (por ejemplo, un CTO o un auditor externo) sin que tenga que interpretar una planilla. Ambos con la posibilidad de filtrar por un rango de fechas.

QUÉ IMPLICA

Del lado del servidor, generar los dos archivos descargables a partir de los mismos datos existentes. Del lado de la pantalla, dos botones de descarga —uno por formato— en el mismo lugar de la interfaz.

POR QUÉ IMPORTA

Los usuarios secundarios del producto (CTOs, responsables de cumplimiento) necesitan poder llevarse los datos a una planilla propia para sus reportes o auditorías, o recibir directamente un documento presentable sin tener que procesar nada — sin esto, dependen completamente de mirar la aplicación en vivo para cualquier informe externo. Van juntas en una sola sección porque comparten exactamente el mismo origen de datos y el mismo caso de uso, y solo cambia el formato de salida.

Reportes por período

SITUACIÓN ACTUAL

La única forma de sacar información agregada del sistema es mirando el Dashboard en el momento, o —una vez implementada la exportación CSV de otra sección— descargando el listado completo de filas sueltas. No existe una vista que resuma cómo estuvo la actividad de seguridad durante un rango de fechas puntual (por ejemplo, "el mes pasado").

QUÉ SE BUSCA

Una vista de reporte donde se pueda elegir un rango de fechas y ver un resumen de ese período: cuántas vulnerabilidades se detectaron, cuántas se resolvieron, y cuál fue el MTTR de esas semanas.

QUÉ IMPLICA

Del lado del servidor, un cálculo que agregue esas métricas dentro del rango de fechas que se le pida. Del lado de la pantalla, un selector de fechas y una vista de resumen con esos números.

POR QUÉ IMPORTA

Es distinto de exportar filas sueltas a CSV — acá se trata de comparar la actividad de un período contra otro, algo que un responsable de seguridad necesita para reportarle a su jefatura sin tener que armar el cálculo a mano en una planilla.

Vista personal "Mis vulnerabilidades asignadas"

SITUACIÓN ACTUAL

El campo de responsable en cada vulnerabilidad es un texto libre y no hay ninguna forma de filtrar el listado general o el Kanban para ver solamente lo que le corresponde a una persona puntual — siempre se ve todo junto.

QUÉ SE BUSCA

Que cada analista, una vez que inicia sesión, pueda ver una vista propia con solo las vulnerabilidades que tiene asignadas, en vez de tener que buscarlas entre todas las demás.

QUÉ IMPLICA

Del lado del servidor, un filtro que devuelva solo las vulnerabilidades asignadas a la persona que está pidiendo la información. Del lado de la pantalla, una vista o un filtro rápido dentro del Kanban o del listado general que muestre "lo mío" con un solo clic.

POR QUÉ IMPORTA

A medida que crece la cantidad de vulnerabilidades detectadas, cada analista termina revisando una lista completa para encontrar lo que realmente le toca a él. Esta vista reduce ese trabajo a lo relevante para cada persona.

Requisito previo: depende de que exista login real (Sección 5), porque hoy no hay forma de saber quién es "yo" dentro del sistema.

Carga de trabajo por responsable

SITUACIÓN ACTUAL

No hay ninguna vista que muestre cuántas vulnerabilidades tiene asignadas cada persona del equipo — la única forma de saberlo hoy sería contar a mano dentro del Kanban.

QUÉ SE BUSCA

Una vista simple que muestre, por cada responsable, cuántas vulnerabilidades tiene asignadas actualmente y cuántas de esas son críticas.

QUÉ IMPLICA

Del lado del servidor, un cálculo que agrupe las vulnerabilidades abiertas por la persona asignada. Del lado de la pantalla, una lista o tabla corta con ese resumen, por ejemplo dentro del Dashboard o de la sección de Configuración.

POR QUÉ IMPORTA

Es distinta del ranking de activos más riesgosos (Sección 19) — ahí se compara activo contra activo, acá se compara persona contra persona. Le da a quien coordina el equipo una forma simple de notar si alguien quedó con mucho más trabajo asignado que el resto.

Acciones en lote sobre vulnerabilidades

SITUACIÓN ACTUAL

Cada cambio de estado o de responsable asignado se hace vulnerabilidad por vulnerabilidad, una por una. Si un scan trae 30 hallazgos nuevos que en realidad corresponden todos a la misma causa, no hay forma de gestionarlos juntos.

QUÉ SE BUSCA

Poder seleccionar varias vulnerabilidades a la vez desde el listado y aplicarles la misma acción en un solo paso — por ejemplo, asignarlas todas a la misma persona, o moverlas todas de estado juntas.

QUÉ IMPLICA

Del lado del servidor, un punto de acceso que reciba una lista de identificadores y aplique el mismo cambio a todos. Del lado de la pantalla, una casilla de selección por fila en el listado y un botón de acción que aparezca cuando hay elementos seleccionados.

POR QUÉ IMPORTA

Ahorra tiempo real en el uso diario del sistema, especialmente después de un scan grande que puede traer muchos hallazgos similares de una sola vez.

Escaneo en lote de activos

SITUACIÓN ACTUAL

Hoy solo se puede disparar un escaneo de a un activo por vez, desde el botón correspondiente en el Inventario. Si se quieren auditar varios activos —o todos los de un entorno— hay que repetir la misma acción manualmente, uno por uno.

QUÉ SE BUSCA

Poder seleccionar varios activos a la vez (o todos los de un entorno puntual) desde el Inventario y disparar el mismo escaneo que ya existe para todos ellos en un solo paso.

QUÉ IMPLICA

Del lado del servidor, repetir el mismo llamado de escaneo que ya se usa para un activo individual, pero para cada uno de los activos seleccionados. Del lado de la pantalla, una casilla de selección por fila en el Inventario y un botón de acción que dispare el escaneo de todos los seleccionados a la vez.

POR QUÉ IMPORTA

Es la misma lógica que ya existe para acciones en lote sobre vulnerabilidades (Sección 25), aplicada esta vez al escaneo de activos —ahorra tiempo real cuando se necesita auditar un conjunto grande de activos de una sola vez, sin depender de repetir la misma acción manualmente.

Búsqueda global

SITUACIÓN ACTUAL

Cada sección de la aplicación (Inventario, Kanban, Entornos) tiene su propio buscador local, que solo busca dentro de esa sección. No hay una forma de buscar algo sin saber primero en qué pantalla está.

QUÉ SE BUSCA

Un buscador único, accesible desde cualquier parte de la aplicación, que encuentre activos, vulnerabilidades y entornos al mismo tiempo, y lleve directo al resultado correspondiente.

QUÉ IMPLICA

Del lado del servidor, un punto de acceso que busque en las distintas tablas del sistema a la vez y devuelva resultados combinados. Del lado de la pantalla, un buscador accesible globalmente (por ejemplo, desde la barra superior o con un atajo de teclado) que muestre esos resultados y navegue al elegido.

POR QUÉ IMPORTA

A medida que crece la cantidad de activos y vulnerabilidades, tener que adivinar en qué sección buscar algo puntual empieza a ser una fricción real —un buscador único la elimina.

Centro de notificaciones dentro de la app

SITUACIÓN ACTUAL

Los avisos del sistema hoy solo llegan por Slack, a través del flujo de n8n. Dentro de la aplicación misma no hay ningún lugar donde ver, de forma pasiva, si pasó algo importante recientemente.

QUÉ SE BUSCA

Una campanita de notificaciones dentro de la aplicación que muestre los eventos recientes más relevantes — por ejemplo, nuevas vulnerabilidades críticas detectadas — sin depender de que alguien esté mirando Slack en ese momento.

QUÉ IMPLICA

Del lado del servidor, un punto de acceso que devuelva los eventos recientes más relevantes ya guardados en el sistema. Del lado de la pantalla, un ícono con contador en la barra de navegación y un desplegable que liste esos eventos.

POR QUÉ IMPORTA

No todo el que usa el sistema tiene por qué estar en el canal de Slack del equipo — esta sección le da al producto un canal de aviso propio, dentro de la misma herramienta, sin depender de un canal externo ni de tocar nada del lado de n8n.

Comparación entre dos escaneos

SITUACIÓN ACTUAL

Con el historial de escaneos ya contemplado (Sección 15), lo único que se podría ver es una lista plana de ejecuciones pasadas, sin ninguna forma de comparar una contra otra.

QUÉ SE BUSCA

Poder elegir dos ejecuciones puntuales del historial y ver la diferencia entre ellas: qué vulnerabilidades son nuevas desde ese momento, cuáles se resolvieron en el medio, y cuáles siguen exactamente igual.

QUÉ IMPLICA

Del lado del servidor, un cálculo que compare los resultados asociados a dos ejecuciones distintas. Del lado de la pantalla, una vista dentro del historial de escaneos donde se puedan seleccionar dos puntos en el tiempo y ver ese resultado comparado.

POR QUÉ IMPORTA

Convierte al historial de una lista de referencia en una herramienta útil para entender la evolución real del riesgo entre dos momentos puntuales, en vez de solo saber que un scan se ejecutó.

Requisito previo: depende de que el historial de escaneos (Sección 15) ya esté implementado.

Exportación del inventario de activos

SITUACIÓN ACTUAL

La exportación de reportes ya contemplada (Sección 21, en CSV y PDF) es solamente de vulnerabilidades. No existe ninguna forma de sacar del sistema la lista de activos en sí —qué software, qué versión, en qué entorno— sin copiarla a mano.

QUÉ SE BUSCA

Poder descargar el inventario completo de activos como un archivo, de forma similar a como se pensó la exportación de vulnerabilidades.

QUÉ IMPLICA

Del lado del servidor, generar ese archivo a partir de los datos de activos ya existentes. Del lado de la pantalla, un botón de descarga dentro del Inventario, en el mismo estilo que el que ya se pensó para las vulnerabilidades.

POR QUÉ IMPORTA

Le da valor a un caso de uso distinto al de las vulnerabilidades — entregarle a alguien de afuera (un auditor, un proveedor) la lista de qué activos existen y dónde viven, sin exponerle el sistema completo ni tener que armar esa lista a mano.

Distribución por ecosistema

SITUACIÓN ACTUAL

El Dashboard ya tiene un gráfico de Distribución por Severidad y otro de Distribución por Estado, pero ninguno muestra cómo se reparten las vulnerabilidades según el ecosistema del activo afectado (npm, pip, maven, etc.), a pesar de que ese dato ya existe en cada activo.

QUÉ SE BUSCA

Un gráfico más, del mismo estilo que los dos que ya existen, que muestre qué proporción de las vulnerabilidades corresponde a cada ecosistema.

QUÉ IMPLICA

Del lado del servidor, agrupar las vulnerabilidades por el ecosistema del activo al que pertenecen. Del lado de la pantalla, un gráfico adicional junto a los que ya están.

POR QUÉ IMPORTA

Le da al Dashboard una dimensión más para entender de dónde viene el riesgo — por ejemplo, si la mayoría de las vulnerabilidades detectadas corresponden a un ecosistema puntual, eso orienta dónde poner atención primero.

Selector de rango de tiempo para todo el Dashboard

SITUACIÓN ACTUAL

El único gráfico con noción de tiempo (la tendencia de detectadas vs. resueltas) está fijo a 30 días sin poder cambiarlo. El resto de las tarjetas y gráficos no tienen ninguna relación con un período — siempre muestran el estado actual o el total histórico.

QUÉ SE BUSCA

Un selector arriba de todo el Dashboard (por ejemplo 7, 30 o 90 días) que afecte a la página completa, no solo al gráfico de tendencia.

QUÉ IMPLICA

Del lado del servidor, que el cálculo de las estadísticas acepte un rango de fechas en vez de tenerlo fijo. Del lado de la pantalla, el selector, y que todos los gráficos y tarjetas se vuelvan a pedir cuando cambia.

POR QUÉ IMPORTA

Es distinto de los "Reportes por período" (Sección 22), que es una vista de reporte aparte pensada para compartir — esto es sobre el Dashboard principal en sí, para poder mirar la misma pantalla de siempre con distintas ventanas de tiempo sin salir de ahí.

Gráficos interactivos con drill-down

SITUACIÓN ACTUAL

Los tres gráficos del Dashboard (tendencia, severidad, estado) son puramente visuales — hacer clic en una porción del gráfico de torta o en una barra no lleva a ningún lado.

QUÉ SE BUSCA

Que hacer clic en una parte de un gráfico (por ejemplo, la porción "CRITICAL" del gráfico de severidad) lleve directamente al listado de vulnerabilidades filtrado por ese mismo criterio.

QUÉ IMPLICA

Es un cambio del lado de la pantalla — conectar cada gráfico con la navegación hacia el listado de vulnerabilidades, aplicando el filtro correspondiente al elemento en el que se hizo clic.

POR QUÉ IMPORTA

Hoy el Dashboard es solo para mirar; con esto pasa a ser también un punto de partida para actuar, sin tener que ir manualmente a buscar lo mismo que ya se estaba viendo en el gráfico.

Requisito previo: aprovecha mejor su potencial una vez que estén los filtros avanzados del listado de vulnerabilidades (Sección 17) — el filtro por estado ya existe hoy, pero el de prioridad y ecosistema todavía no.

Caché del Dashboard con Caffeine

SITUACIÓN ACTUAL

Cada vez que alguien abre o recarga el Dashboard, el sistema ejecuta ocho consultas distintas a la base de datos para recalculer absolutamente todas las estadísticas desde cero, aunque nada haya cambiado desde la última vez que se cargó.

QUÉ SE BUSCA

Que ese cálculo se guarde en memoria por un tiempo corto (alrededor de un minuto) y se reutilice si alguien vuelve a pedir el Dashboard dentro de esa ventana, en vez de recalcularlo todo de nuevo cada vez.

QUÉ IMPLICA

Es un cambio puramente del lado del servidor — no requiere ningún ajuste visible en la pantalla, el usuario ve exactamente lo mismo, solo que se genera con menos trabajo de base de datos detrás.

POR QUÉ IMPORTA

Es una mejora de eficiencia pura — reduce la carga innecesaria sobre la base de datos generada por algo que no necesita recalcularse en cada segundo.

Auto-actualización periódica del Dashboard

SITUACIÓN ACTUAL

Los datos del Dashboard se cargan una sola vez al entrar a la página. Si alguien la deja abierta —por ejemplo como pantalla de estado en una oficina— los números quedan desactualizados hasta que alguien la recargue a mano.

QUÉ SE BUSCA

Que el Dashboard se actualice solo cada cierto tiempo, sin necesidad de que alguien recargue la página.

QUÉ IMPLICA

Es un cambio únicamente del lado de la pantalla — repetir el pedido de estadísticas cada cierto intervalo mientras la página sigue abierta.

POR QUÉ IMPORTA

Le permite al Dashboard funcionar como una pantalla de estado en vivo, algo natural para una herramienta de seguridad pensada para tener siempre a la vista.

Manejo de error con reintento visible

SITUACIÓN ACTUAL

Si falla la carga de las estadísticas del Dashboard, hoy se muestra un aviso de error (toast) pero la página queda con todos los espacios vacíos para siempre, sin ninguna forma de reintentar sin recargar toda la página.

QUÉ SE BUSCA

Que, si falla la carga, la página muestre un mensaje claro de que algo salió mal junto con un botón para reintentar, en lugar de quedar con espacios vacíos sin explicación.

QUÉ IMPLICA

Es un cambio únicamente del lado de la pantalla — manejar el caso de error de forma explícita en vez de dejar la página en un estado a medio cargar.

POR QUÉ IMPORTA

Sin esto, si el Dashboard falla al cargar una vez, la única forma de recuperarse es que el usuario se dé cuenta por su cuenta de que tiene que recargar toda la página.

Mejoras visuales del Kanban

SITUACIÓN ACTUAL

Las tarjetas del Kanban muestran información mínima. No hay ninguna señal visual que distinga una vulnerabilidad que está siendo explotada activamente (confirmada en el catálogo CISA KEV) de una que no; las fechas se muestran como timestamp exacto en vez de algo más legible como "hace 2 días"; no hay ninguna alerta sobre tarjetas que llevan mucho tiempo estancadas en "En análisis"; y al arrastrar una tarjeta entre columnas no hay ninguna señal visual de dónde va a caer.

QUÉ SE BUSCA

Agregar una marca visual distintiva en las tarjetas de vulnerabilidades confirmadas como explotadas activamente; mostrar la fecha de detección en formato relativo y legible; resaltar de alguna forma las tarjetas que llevan demasiados días sin resolver en "En análisis"; y mostrar una zona de destino claramente marcada mientras se arrastra una tarjeta.

QUÉ IMPLICA

Son todos cambios dentro de la pantalla del Kanban — la información que necesitan ya está disponible, se trata de presentarla de forma más clara y con más contexto visual.

POR QUÉ IMPORTA

Ayuda al analista a priorizar de un vistazo qué tarjetas necesitan atención urgente, sin tener que abrir cada una para enterarse.

Sidebar y navegación

SITUACIÓN ACTUAL

El menú lateral ocupa un ancho fijo sin posibilidad de achicarse, no hay ninguna señal en el menú de cuántas vulnerabilidades están pendientes sin entrar al Kanban, y el pie de página solo muestra un indicador genérico de "sistema activo".

QUÉ SE BUSCA

Que el menú lateral se pueda colapsar a un modo compacto que solo muestre íconos (liberando espacio para el contenido principal); que aparezca un número sobre el ícono del Kanban indicando cuántas vulnerabilidades están en estado "Detectada" sin revisar; y que el pie de página muestre la fecha y hora del último escaneo ejecutado en vez de un indicador genérico.

QUÉ IMPLICA

Son todos cambios de navegación y presentación — no requieren datos nuevos que no existan ya en el sistema.

POR QUÉ IMPORTA

Son detalles de navegación que hacen que la aplicación se sienta más prolija y dan una noción constante de trabajo pendiente sin tener que entrar a buscarlo.

Mejoras globales de UI/UX

SITUACIÓN ACTUAL

La aplicación es completamente oscura sin posibilidad de cambiarlo; los mensajes de confirmación (toasts) son genéricos y no llevan a ningún lado; las tablas vacías no explican qué hacer a continuación; borrar un elemento usa el cuadro de confirmación nativo del navegador; las animaciones de entrada solo están aplicadas en el Dashboard; y el título de la pestaña del navegador es siempre el mismo sin importar en qué sección se esté.

QUÉ SE BUSCA

Agregar un selector de tema claro/oscuro que se recuerde entre sesiones; que los mensajes de confirmación incluyan una acción clicable cuando corresponda (por ejemplo, "Activo actualizado — Ver en inventario"); reemplazar las tablas vacías por un mensaje con una guía de qué hacer y un botón de acción; reemplazar el cuadro de confirmación nativo del navegador por uno propio con el nombre del elemento a eliminar; extender la animación de entrada que ya existe en el Dashboard a todas las páginas; y que el título de la pestaña cambie según la sección activa, mostrando un aviso si hay vulnerabilidades críticas pendientes.

QUÉ IMPLICA

Son mejoras transversales que tocan varias pantallas a la vez, más que una sección puntual del sistema.

POR QUÉ IMPORTA

Son el tipo de detalles que hacen que un producto se sienta terminado y profesional — algo que pesa especialmente en una demostración en vivo frente a un tribunal de tesis.

